SENTINELAI · SECURITY ASSESSMENT REPORT

example.com

Quarterly External Assessment · bug_bounty · 2026-10-11 14:30 UTC

Authorization: Authorized under internal pentest engagement PT-2026-Q4. | **By:** security@example.com

Executive Summary

SentinelAI performed an automated, AI-verified security assessment of **example.com**. The scan mapped the target's attack surface and ran vulnerability checks whose results were then independently reviewed by multiple AI models, with only cross-validated findings promoted into this report to minimise false positives.

Overall Risk

Critical

Risk score: 90/100

1	2	1	1	1
critical	high	medium	low	info

Findings

1. SQL Injection in product search

critical

Category	Injection
Endpoint	GET https://api.example.com/v1/products
Parameter	q
CWE	CWE-89
OWASP	A03:2021 – Injection
CVSS band	9.0 – 10.0

The product search endpoint concatenates the 'q' parameter directly into a SQL query. A boolean-based blind injection was confirmed: payloads altering the WHERE clause changed the result set deterministically, and a time-based payload induced a measurable server delay.

Evidence

```
GET /v1/products?q=widget' AND 1=1-- - → 200, 42 results
```

```
GET /v1/products?q=widget' AND 1=2-- - → 200, 0 results
```

```
GET /v1/products?q=widget';SELECT pg_sleep(5)-- - → 5.1s response
```

Remediation

Replace string concatenation with parameterized queries / prepared statements. Add an allow-list validator for the search term and apply least-privilege database credentials so the web role cannot read unrelated tables.

References

- https://owasp.org/www-community/attacks/SQL_Injection
- https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

AI Consensus

Verdict: **Confirmed** | Agreement: **100%** | Confidence: **94%**

Models: gemini, deepseek, llama3.1 (local)

2. Reflected Cross-Site Scripting in error page

high

Category	Cross-Site Scripting
Endpoint	GET https://www.example.com/search
Parameter	query
CWE	CWE-79
OWASP	A03:2021 – Injection
CVSS band	7.0 – 8.9

The search results page reflects the 'query' parameter into the HTML response without output encoding, allowing arbitrary script execution in a victim's browser.

Evidence

```
GET /search?query=<script>alert(document.domain)</script>
```

```
Response body contained the payload unescaped inside <h2>…</h2>
```

Remediation

Context-aware output encoding on all reflected values, a strict Content-Security-Policy, and framework auto-escaping (do not disable it for this view).

References

- https://owasp.org/www-community/attacks/xss/

AI Consensus
Verdict: **Confirmed** | Agreement: **100%** | Confidence: **89%**
Models: gemini, deepseek, llama3.1 (local)

3. Insecure Direct Object Reference on invoice download

high

Category	`Broken Access Control`
Endpoint	`GET https://api.example.com/v1/invoices/{id}`
Parameter	`id`
CWE	`CWE-639`
OWASP	`A01:2021 – Broken Access Control`
CVSS band	`7.0 – 8.9`

Invoice IDs are sequential integers and the endpoint does not verify that the authenticated user owns the requested invoice, allowing horizontal privilege escalation and disclosure of other customers' billing data.

Evidence

```
User A (invoice 1001) requested /v1/invoices/1002 → 200 with User B's data
```

Remediation

Enforce an ownership check on every object access, and switch sequential IDs to unguessable UUIDs as defense in depth.

References

- https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html

AI Consensus
Verdict: **Likely** | Agreement: **75%** | Confidence: **78%**
Models: gemini, deepseek, llama3.1 (local)

4. Missing security headers

medium

Category	`Security Misconfiguration`
Endpoint	`GET https://www.example.com/`
CWE	`CWE-693`
OWASP	`A05:2021 – Security Misconfiguration`

CVSS band	4.0 – 6.9

Responses omit Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options and X-Frame-Options, weakening defense-in-depth against XSS, clickjacking and protocol-downgrade attacks.

Evidence

```
Response headers lacked: Content-Security-Policy, Strict-Transport-Security, X-Frame-Options
```

Remediation

Add HSTS (with preload), a restrictive CSP, X-Content-Type-Options: nosniff and X-Frame-Options: DENY at the edge/reverse proxy.

References

- https://owasp.org/www-project-secure-headers/

AI Consensus
Verdict: **Confirmed** | Agreement: **100%** | Confidence: **82%**
Models: gemini, deepseek, llama3.1 (local)

5. Open redirect via 'next' parameter

low

Category	Open Redirect
Endpoint	GET https://www.example.com/login
Parameter	next
CWE	CWE-601
OWASP	A01:2021 – Broken Access Control
CVSS band	0.1 – 3.9

The login flow redirects to an attacker-controlled absolute URL supplied in 'next' after authentication, enabling phishing and OAuth token theft chains.

Evidence

```
GET /login?next=https://evil.example.net → 302 Location: https://evil.example.net
```

Remediation

Allow only relative paths or an explicit allow-list of hosts for post-login redirects.

References

- https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html

AI Consensus
Verdict: **Likely** | Agreement: **66%** | Confidence: **61%**
Models: gemini, deepseek, llama3.1 (local)

6. Verbose server banner discloses software version

info

Category	`Information Disclosure`
Endpoint	`GET https://staging.example.com/`
CWE	`CWE-200`
OWASP	`A05:2021 – Security Misconfiguration`
CVSS band	`0.0`

The Server header and default error pages reveal exact software versions, helping an attacker target known CVEs.

Evidence

```
Server: Apache/2.4.57 (Ubuntu)
```

Remediation

Suppress or genericize version banners (ServerTokens Prod).

AI Consensus
Verdict: **Confirmed** | Agreement: **100%** | Confidence: **70%**
Models: gemini, deepseek, llama3.1 (local)

Attack Surface

Subdomains: www.example.com, api.example.com, staging.example.com, blog.example.com, mail.example.com

Host	IPs	Technologies	Open Ports
`www.example.com`	93.184.216.34	nginx 1.25.3, React 18, Cloudflare	80, 443
`api.example.com`	93.184.216.35	nginx 1.25.3, FastAPI, PostgreSQL	443, 8080
`staging.example.com`	93.184.216.36	Apache 2.4.57, PHP 8.1, MySQL	80, 443, 22

Methodology & Tooling

Tools: subfinder, dnsx, httpx, naabu, katana, ffuf, nuclei
Pipeline: Reconnaissance → DNS → HTTP Discovery → Port Scan → Crawl → Directory Scan → Nuclei → Multi-Model AI Consensus → Reporting.

This report was generated by SentinelAI for an authorized security assessment. Findings are AI-assisted and should be validated by a qualified security professional before remediation decisions. Unauthorized scanning of systems you do not own or have explicit written permission to test is illegal.